

⚡ LLMOPS & AI ENGINEERING PREP

15+ Fine-Tuning Interview Questions

📖 WHAT YOU'LL MASTER INSIDE

Every Answer Explained Simply

SFT • Instruction Tuning • Catastrophic Forgetting • LoRA & QLoRA •
Adapters & Trade-offs



Check caption for complete AI Engineer Interview Kit.

Q1. What is fine-tuning in LLMs, and how does it differ from pretraining?

Pretraining teaches language & world knowledge from hundreds of billions of raw internet tokens. Fine-tuning shapes a pretrained model on curated task examples.

Aspect	Pretraining	Fine-Tuning
Data Size	Hundreds of billions of tokens	Thousands to millions of tokens
Compute	Thousands of GPUs for weeks	Single GPU to small cluster (hours)
Goal	Learn language & world knowledge	Adapt behavior for specific task
Who Does It	Model providers (OpenAI, Meta)	Application engineering teams
Starting Point	Random weight initialization	Pretrained base model weights

PRACTICAL ANALOGY

Pretraining is **20 years of general education**. Fine-tuning is a **3-month specialized training program** after they are already educated. The general knowledge is already there; you are shaping how they apply it.

 Check caption for complete AI Engineer Interview Kit.

Q2. When would you choose fine-tuning over prompt engineering?

Try prompt engineering first. Fine-tuning carries real costs in data prep, compute, and maintenance—only justified when prompting genuinely hits a wall.

✓ CHOOSE FINE-TUNING WHEN

- ✓ **Consistent format at scale:** Eliminate 1,500 tokens of few-shot examples per request.
- ✓ **Conflict with base training:** Override default persona or refusal patterns reliably.
- ✓ **Domain specialization:** Internal terminology or custom DSL syntax.
- ✓ **Cost reduction:** Fine-tuned small model matching larger base quality at lower latency.

⚠ STAY WITH PROMPTING WHEN

- ⚠ **Rapidly evolving specs:** Requirements change every few weeks.
- ⚠ **Low traffic volume:** Token overhead cost is negligible.
- ⚠ **General-purpose tasks:** Broad reasoning without rigid schema.
- ⚠ **Prompt already works:** Meets production quality bar.

📈 THE QUALITY & COST CEILING RULE

Fine-tuning pushes past the quality ceiling when advanced prompting techniques (CoT, few-shot, role prompting) plateau, or when token economies require distilling GPT-4o quality into GPT-4o mini.

🔖 Check caption for complete AI Engineer Interview Kit.

Q3. Compare prompting, RAG, and fine-tuning. When is each appropriate?

These approaches solve distinct problems and are complementary in production systems rather than mutually exclusive alternatives.

PROMPTING

Role: Guidance at inference.

Best for: General tasks & fast iteration.

RAG

Role: External knowledge.

Best for: Fresh facts, docs & traceability.

FINE-TUNING

Role: Weight updates.

Best for: Style, format & domain syntax.

Production Scenario	Recommended Architecture
General Assistant	Prompting only
Support over Knowledge Base	RAG + Prompting
Consistent Legal Document Formatting	Fine-Tuning + Prompting
Medical Q&A with Proprietary Protocols	Fine-Tuning + RAG + Prompting
Code Generation in Internal DSL	Fine-Tuning

PRODUCTION BEST PRACTICE

Use **Fine-Tuning** for form, style & domain syntax, **RAG** for accurate real-time factual retrieval, and **Prompting** for query-specific execution.

 Check caption for complete AI Engineer Interview Kit.

Q4. What is Supervised Fine-Tuning (SFT), and how is training data structured?

SFT trains on curated input-output pairs using cross-entropy loss to maximize the probability of generating ideal target responses.

```
{ "messages": [ { "role": "system", "content": "You are a contract analyst." }, { "role": "user", "content": "Summarize clause: The vendor shall indemnify..." }, { "role": "assistant", "content": "Vendor protects client from legal claims..." } ] }
```

✓ DATA QUALITY RULES

- **High-signal target:** Outputs mirror exact ideal format.
- **Broad diversity:** Cover edge cases seen in prod.
- **Structural consistency:** Similar inputs get uniform schemas.
- **Volume target:** 1k–10k examples for task adaptation.

📦 SOURCING STRATEGIES

- ✓ **Human annotators:** Gold-standard quality, highest cost.
- ✓ **GPT-4 distillation:** Scalable synthetic generation + human QA.
- ✓ **Production logs:** Filtered golden traces from existing workflows.



Check caption for complete AI Engineer Interview Kit.

Q5. What is instruction tuning, and how does it improve model alignment?

Instruction tuning trains a base model on diverse instruction-response pairs, teaching it to follow commands reliably rather than merely completing text.

✗ RAW PRETRAINED BASE MODEL

Acts as a statistical text completer.
Input: "What is the capital of France?"
Output: "...a question often asked by geography students."

✓ INSTRUCTION-TUNED MODEL

Understands intent as an actionable task.
Input: "What is the capital of France?"
Output: "The capital of France is Paris."

WHY TASK DIVERSITY MATTERS (FLAN / INSTRUCTGPT)

Training across hundreds of task families (summarization, translation, coding, QA) teaches generalized instruction adherence. The model generalizes to unseen instruction types zero-shot.

ALIGNMENT & HELPFULNESS

Beyond task execution, instruction tuning establishes baseline alignment: answering directly, maintaining helpful tone, and respecting user constraints before RLHF refinement.

 Check caption for complete AI Engineer Interview Kit.

Q6. What are the risks of full-parameter fine-tuning on large models?

Updating all 70B+ parameters carries significant computational, architectural, and behavioral risks that make it impractical for most teams.



1. CATASTROPHIC FORGETTING

Updating all weights overwrites representations learned during pretraining. Coding specialization can degrade reasoning capabilities.

\$ 2. COMPUTE & COST

Full gradients for a 70B model require dozens of A100/H100 GPUs (\$10k+ per run). Hyperparameter errors are extremely costly.



3. OVERFITTING RISK

Small datasets relative to 70B parameters cause memorization. The model excels on training prompts but fails on slight variations.



4. SERVING COMPLEXITY

Produces a distinct multi-gigabyte model per task requiring dedicated GPUs—no shared infrastructure across model variants.



Check caption for complete AI Engineer Interview Kit.

Q7. What is catastrophic forgetting in fine-tuned models?

When weight updates for a new task overwrite neural representations encoding previously learned general reasoning and world knowledge.

❗ CONCRETE PRODUCTION EXAMPLE

Fine-tuning GPT-4o on customer support data in a specific polite tone. After fine-tuning, support answers are perfect, but complex Python coding and multi-step math capabilities have severely degraded.

🛡️ 4 KEY MITIGATIONS

- 1 **Regularization (EWC):** Penalizes large weight updates to parameters important for prior pretraining tasks.
- 2 **Mixed Training:** Replay a sample of general instruction-following dataset alongside domain data.
- 3 **PEFT / LoRA:** Freeze base weights completely; train lightweight adapters so base representations stay intact.
- 4 **Small Learning Rate:** Minimize weight displacement to reduce interference with pretrained features.

🔖 Check caption for complete AI Engineer Interview Kit.

Q8. How does domain adaptation differ from task-specific fine-tuning?

Domain adaptation immerses a model in specialized terminology; task-specific fine-tuning teaches output structure and execution rules.

DOMAIN ADAPTATION

- ✓ **Goal:** Domain fluency & vocabulary.
- ✓ **Data:** Unstructured corpora (PubMed papers, legal filings, SEC reports).
- ✓ **Objective:** Continued pretraining (next-token prediction).
- ✓ **Result:** Understands cardiology terms like a cardiologist.

TASK-SPECIFIC FINE-TUNING

- ✓ **Goal:** Consistent output formatting.
- ✓ **Data:** Labeled input-output pairs demonstrating ideal responses.
- ✓ **Objective:** Supervised instruction learning.
- ✓ **Result:** Generates strict JSON summaries of clauses.

TWO-STAGE PRODUCTION RECIPE

First perform **Domain Adaptation** so the model comprehends technical literature, then perform **Task Fine-Tuning** to enforce exact production schema compliance.



Check caption for complete AI Engineer Interview Kit.

Q9. What are the trade-offs between Full Fine-Tuning and PEFT?

PEFT trains 0.1%–1% of parameters while freezing base weights, offering massive compute savings with minimal quality drop.

Dimension	Full Fine-Tuning	PEFT (LoRA / Adapters)
Parameters	100% updated (70B params)	0.1% to 1% (~10M–50M params)
Memory Need	280GB+ weights/grads/opt	Single GPU (24GB–48GB w/ QLoRA)
Forgetting Risk	High (base weights overwritten)	Near-Zero (base model frozen)
Serving Model	Dedicated full copy per task	Shared base + hot-swap adapters
Expressiveness	Maximum capacity ceiling	Slightly limited on extreme shifts

✓ PRACTICAL RECOMMENDATION

Use **PEFT / LoRA by default**. The 10x compute reduction and hot-swappable serving architecture far outweigh minor expressiveness differences for 98% of production apps.

🔖 Check caption for complete AI Engineer Interview Kit.

Q10. What signals indicate that fine-tuning may NOT be the right solution?

Fine-tuning requires significant infrastructure commitment. Recognizing these 6 warning indicators saves engineering months.

- 1 Prompting Not Exhausted:** Haven't thoroughly tested few-shot, Chain-of-Thought, or structured output schemas.
- 2 Factual Knowledge Gap:** Model lacks internal docs/catalogs. Fine-tuning causes confident hallucination—use RAG instead.
- 3 Rapidly Evolving Specs:** Output schema or business rules change frequently. Fine-tuned models go stale quickly.
- 4 Dataset < 500 Examples:** Too few high-quality labeled samples leads to severe overfitting and noise memorization.
- 5 No Automated Eval Framework:** Without golden test suites, you cannot verify whether fine-tuning improved or regressed quality.
- 6 Prompting Already Hits 85%+:** Marginal gain rarely justifies full fine-tuning pipeline maintenance costs.



Check caption for complete AI Engineer Interview Kit.

Q11. What is LoRA, and how does it reduce training cost?

LoRA (Low-Rank Adaptation) freezes base model weights W and injects trainable low-rank decomposition matrices A and B in parallel.

$$W' = W_{\text{frozen}} + (B_{r \times d} \times A_{d \times r})$$

⚙️ MECHANICS

- ✓ **$W (d \times d)$:** Base matrix stays frozen.
- ✓ **$A (d \times r)$:** Randomly initialized rank- r matrix.
- ✓ **$B (r \times d)$:** Zero-initialized rank- r matrix.
- ✓ **Rank r :** Typically 4, 8, or 16.

📊 PARAMETER MATH ($R=8$)

- 4096×4096 matrix = **16.7M params**
- LoRA adds $(4096 \times 8) + (8 \times 4096) =$
65,536 params
- Only **0.4%** trainable weights!

⚡ ZERO INFERENCE OVERHEAD

At inference time, matrix product $(B \times A)$ is merged directly into W ($W + BA$), adding zero latency while enabling instant hot-swapping between adapter weights.



Check caption for complete AI Engineer Interview Kit.

Q12. How does QLoRA differ from standard LoRA?

QLoRA combines LoRA with aggressive 4-bit quantization, enabling fine-tuning of 70B parameter models on a single 48GB consumer GPU.

- 1 4-bit NormalFloat (NF4):** Quantizes base weights into an information-theoretically optimal format for normally distributed model parameters.
- 2 Double Quantization:** Quantizes the quantization scale constants themselves, saving roughly 0.5 bits per parameter (~3GB extra savings on 65B+ models).
- 3 Paged Optimizers:** Automatically pages optimizer states to CPU RAM during memory spikes to prevent CUDA out-of-memory crashes.

PRODUCTION IMPACT

Reduces 70B model footprint from **140GB to ~35GB**. Retains **95%–99%** of full-precision LoRA quality while democratizing local GPU adaptation.



Check caption for complete AI Engineer Interview Kit.

Q13. What are adapter layers, and how do they work in practice?

Adapter layers are compact bottleneck neural network modules inserted sequentially between transformer attention and feed-forward blocks.

Input → LayerNorm → Down-proj (d→r) → ReLU → Up-proj (r→d) → +Residual

🔧 ARCHITECTURE

- **Bottleneck dim r:** 64 to 256.
- **Parameters:** ~500k per layer.
- **Initialization:** Up-projection set to near-zero (starts as identity function).

🔄 LEARNING DYNAMICS

- ✓ Base representations flow unchanged at epoch 0.
- ✓ Adapters learn incremental feature shifts.
- ✓ Base weights remain 100% frozen.

⚖️ WHY LORA SUPERSEDED CLASSIC ADAPTERS

Classic adapters add a **serial computation path** that increases inference latency. LoRA updates parallel weight matrices that merge at inference for **zero overhead**.



Check caption for complete AI Engineer Interview Kit.

Q14. How does freezing base model weights impact stability?

Freezing base weights is the architectural pillar of PEFT stability, protecting pretrained knowledge and simplifying training optimization.

- 1 Eliminates Catastrophic Forgetting:** Base representations cannot drift; general capabilities and reasoning are permanently preserved.
- 2 Prevents Gradient Explosions:** Small adapter parameter space remains stable under standard learning rates without gradient overflow.
- 3 Strong Inductive Regularization:** Fewer trainable parameters prevent overfitting and memorization on small domain datasets.
- 4 Enables Higher Learning Rates:** Safely train small random matrices at $1e-4$ vs. fragile $1e-6$ full fine-tuning rates.
- 5 Operational Safety:** If a training experiment diverges, discard the small adapter without reloading base model infrastructure.



Check caption for complete AI Engineer Interview Kit.

Q15. When would you use PEFT instead of Full Fine-Tuning?

Use PEFT almost always. Operational efficiency, multi-tenant serving, and compute savings dominate marginal expressiveness gains.

✓ CHOOSE PEFT / LORA WHEN

- ✓ **Limited GPU budget:** Single 24–48GB GPU.
- ✓ **Multi-tenant serving:** Host 1 shared base + 50 hot-swappable adapter files (~100MB each).
- ✓ **Rapid iteration:** Test 10 LoRA ranks in the time of 1 full training run.
- ✓ **Preserving reasoning:** Maintain base intelligence.

⚠ CHOOSE FULL FINE-TUNING WHEN

- **Maximum quality ceiling:** 2% gain justifies \$20k+ compute on a stable core task.
- **Massive datasets:** >100k high-quality samples where full expressiveness is fully leveraged.

👉 EXECUTIVE RULE OF THUMB

Start with **QLoRA/LoRA**. Only graduate to full fine-tuning if LoRA hits an expressiveness ceiling documented on automated evaluation benchmarks.



Check caption for complete AI Engineer Interview Kit.

Fine-Tuning Architecture Decision Tree

Use this production roadmap to pick the optimal customization technique for your LLMOps engineering stack.

Problem Requirement	Optimal Technical Intervention
Ambiguous Output / Format	Prompt Engineering + Few-Shot Examples
Missing Fresh Factual Data	RAG (Vector Index + Hybrid Search)
Fixed JSON Schema / Tone at Scale	LoRA / QLoRA Task Fine-Tuning
Technical Vocabulary (Medical/Legal)	Domain Adaptation (Continued Pretraining)
Multi-Tenant Custom Assistants	Shared Base Model + Swappable LoRA Adapters

GOLDEN RULES FOR INTERVIEW SUCCESS

- ✓ Never suggest retraining as your first response to quality drop.
- ✓ Explain LoRA rank decomposition **W + BA** clearly.
- ✓ Emphasize evaluation frameworks before data collection.

 Check caption for complete AI Engineer Interview Kit.